

C Programming

6_pointers

```
int scores[3] = {77, 43, 100};  
char ch = 'a';
```

	[0]			[1]	[2]			
Value	...	...	77	43	100	...	'a'	...
Address	0	200	204	208		400	429	4967296

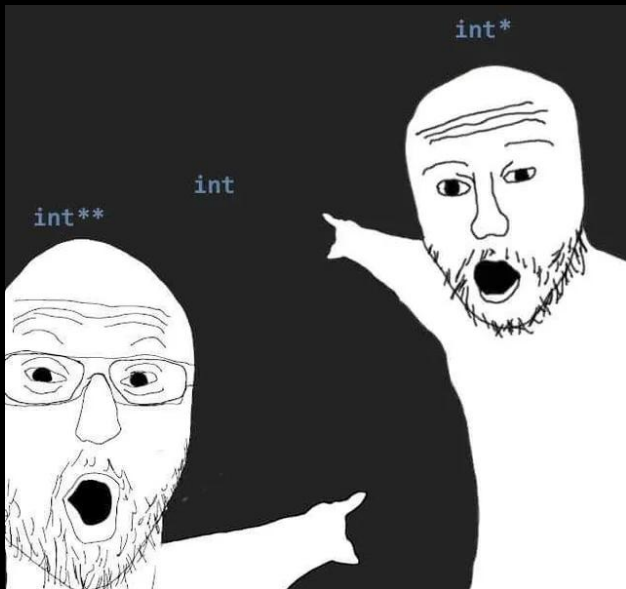
Память похожа на очень длинный массив байт, где индексы этого массива - есть адреса (строка с Address), а значения массива - значения, что хранятся в памяти (строка Value)

`&scores[0], &scores[1], &scores[2]`

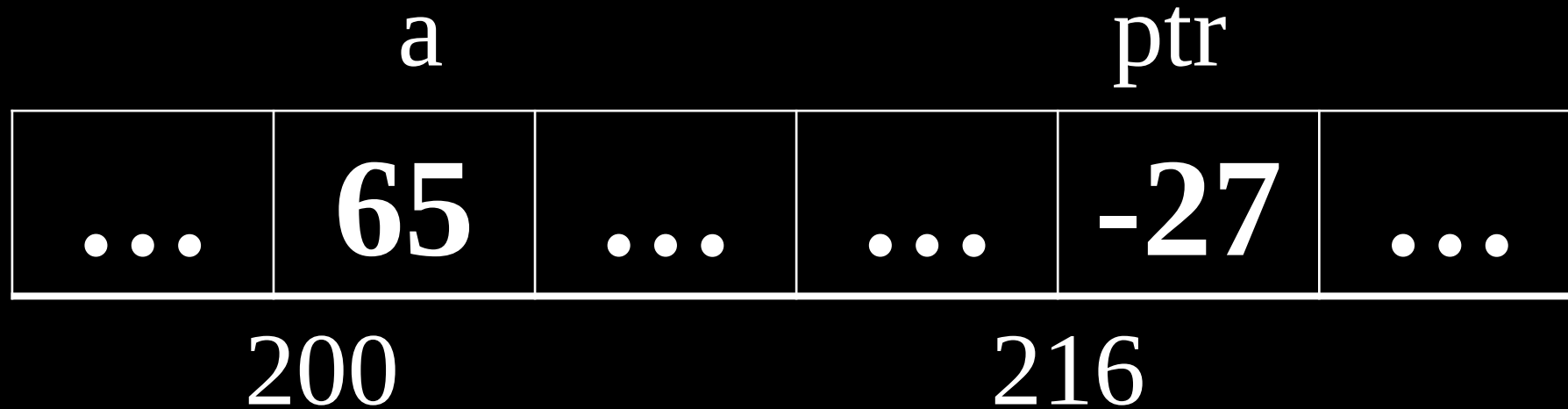
Адрес - номер в памяти с нуля, обозначающийся целым числом. ``&`` - операция взятия адреса переменной

Кстати, `scores == &scores[0]`

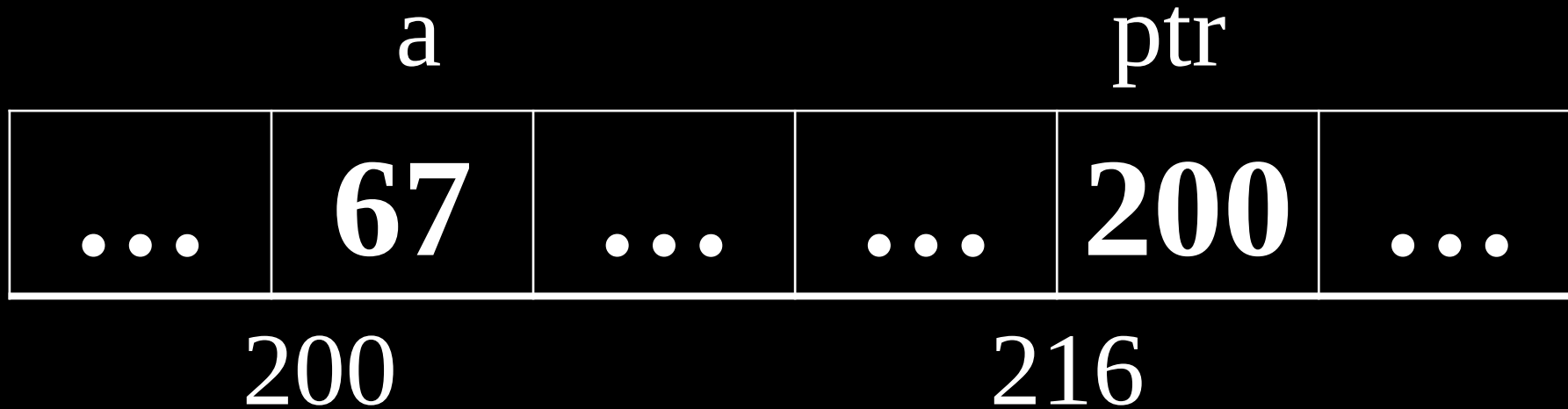
Указатель (pointer) - это переменная, которая хранит адрес ячейки памяти.



```
char a = 'A'; // 65  
char *ptr;
```




```
char a = 'A'; // 65
char *ptr;
ptr = &a;
*ptr = 67;
```



Разыменовыванием (dereference) - называется переход по адресу, что хранится в переменной-указателе для считывания или записи значения по указанному адресу

Здесь не путаем `\*` когда объявляем и когда разыменовываем переменную, это разные вещи:

- `int *ptr;` - `\*` здесь значит, что мы объявляем указатель
- `*ptr = 5;` - `\*` здесь означает операцию разыменовывания

При объявлении указателя есть **ТОЛЬКО ДВА** варианта где поставить звездочку:

- `int *ptr;`
- `int* ptr;`

Void pointers

```
void b = 5; // Ошибка!
```

```
void *ptr; // Указатель на пустоту
```

```
sizeof(char *) == sizeof(void *) == sizeof(int *)
```

Для 32-разрядных систем размер указателя будет 4 байта, для 64 = 8 байт.

void указатель нельзя разыменовывать, если явно не преобразовать его на указатель другого типа

```
int a = 5;  
void *ptr = &a;  
*ptr = 3; // ошибка!
```

```
int a = 5;  
void *ptr = &a;
```

```
char b = *((char *)ptr);
```

NULL

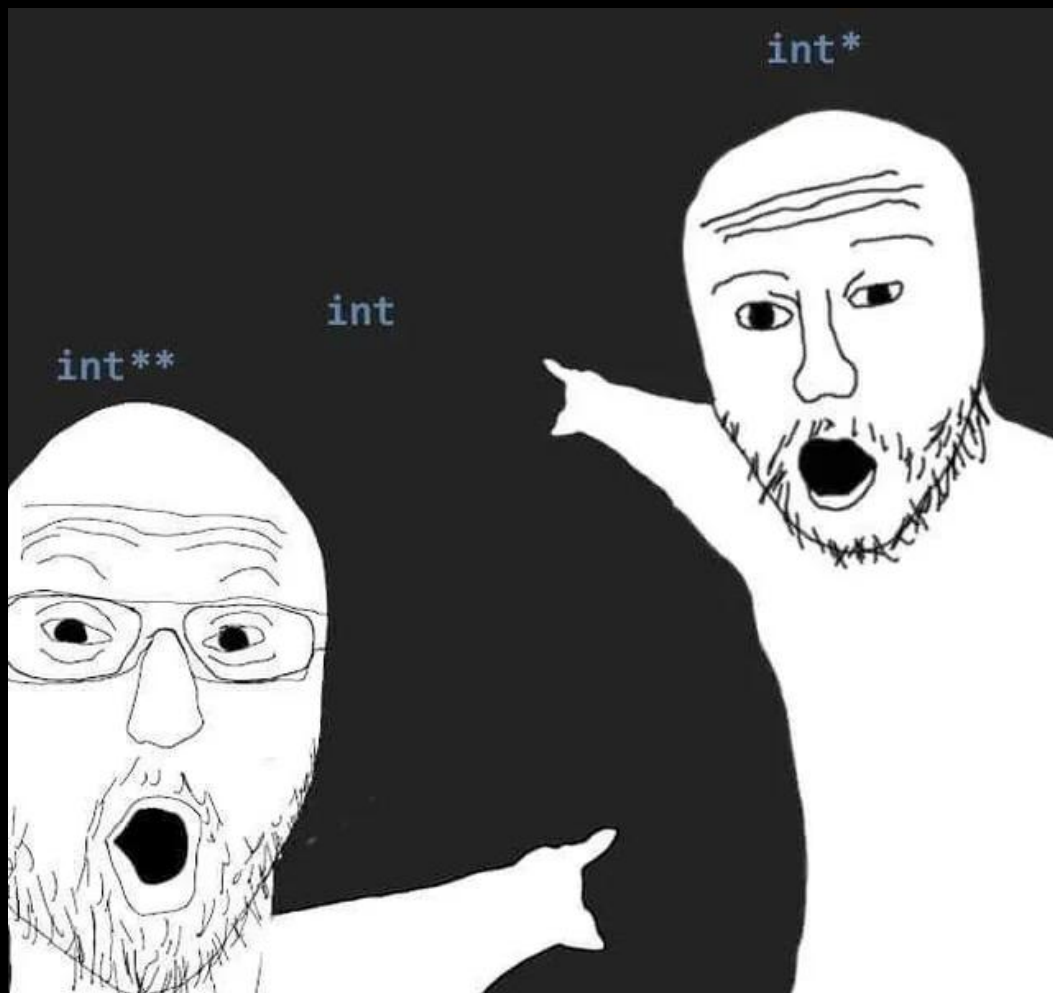
Для указателей есть свой 0 - NULL. NULL - это адрес на нулевую ячейку к памяти. Всем программам к ней доступ закрыт и при попытке разыменовать NULL получим segfault.

```
int *ptr = NULL;
// ..
if (ptr != NULL) { // ptr инициализирован?
    // Что-то выполняем
}
```

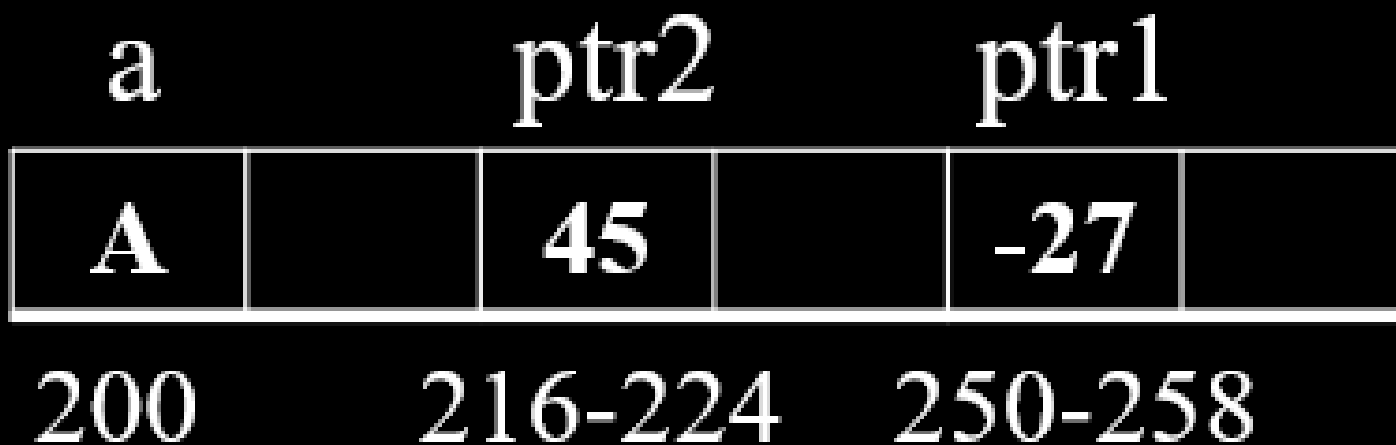
Что здесь объявлено?

```
int* a, b;
```

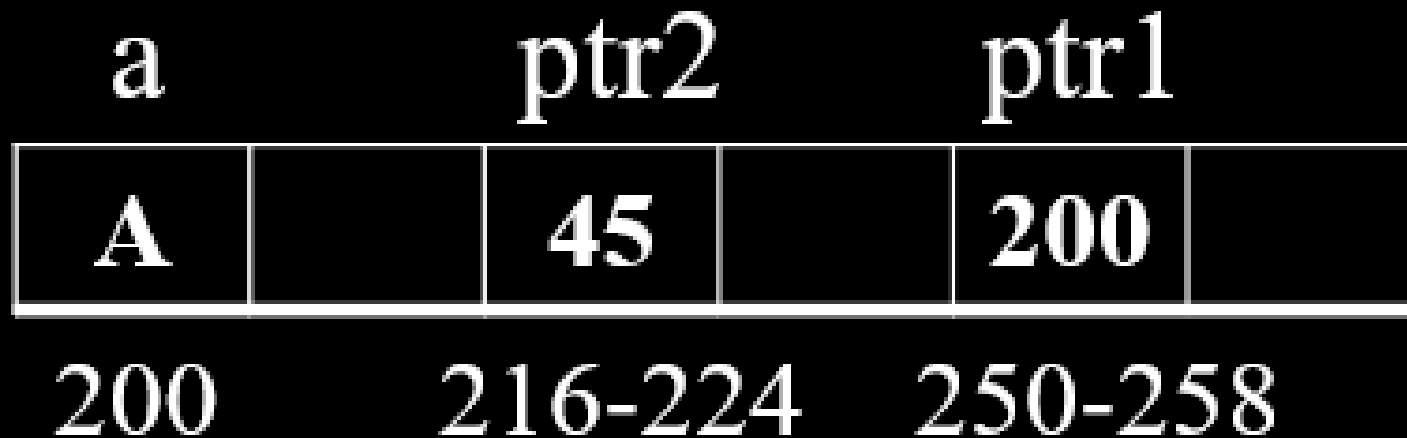
Указатель на указатель на ..



```
char a = 'A';  
char *ptr1;  
char **ptr2; // указатель на указатель на int
```



```
char a = 'A';  
char *ptr1;  
char **ptr2; // указатель на указатель на int  
  
ptr1 = &a;
```



```
char a = 'A';  
char *ptr1;  
char **ptr2; // указатель на указатель на int
```

```
ptr1 = &a;  
ptr2 = &ptr1;
```




```

printf("ptr2 = %p\n", ptr2);           // 250
printf("*ptr2 = %p\n", *ptr2);         // 200
printf("**ptr2 = %c\n", **ptr2);       // A
printf("&ptr = %p\n", &ptr2);          // 216

```

a		ptr2		ptr1	
A		250		200	
200		216-224		250-258	

```
int a = 123;  
int *ptr = &a;  
  
*&*&*&*ptr = ?
```

Pointers to pointers

```
char a = 'A';  
char *ptr1;  
char **ptr2;
```

```
ptr1 = &a;
```

```
ptr2 = &ptr1;  
printf("ptr2 = %p\n", ptr2);      // 250  
printf("*ptr2 = %p\n", *ptr2);    // 200  
printf("**ptr2 = %c\n", **ptr2);  // A  
printf("&ptr = %p\n", &ptr2);     // 216
```

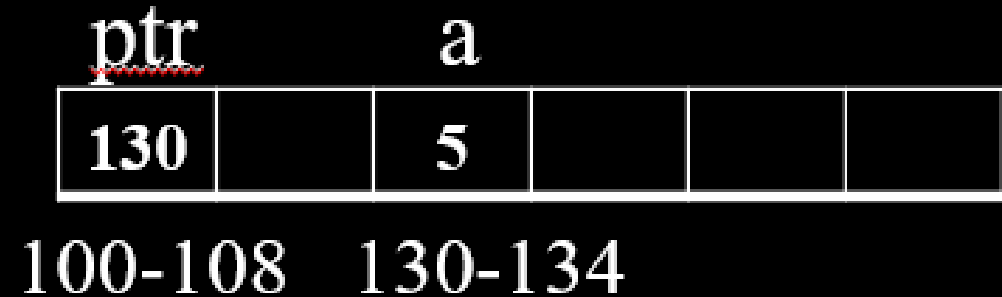
a		ptr2		ptr1	
A		45		-27	
200		216-224		250-258	

a		ptr2		ptr1	
A		45		200	
200		216-224		250-258	

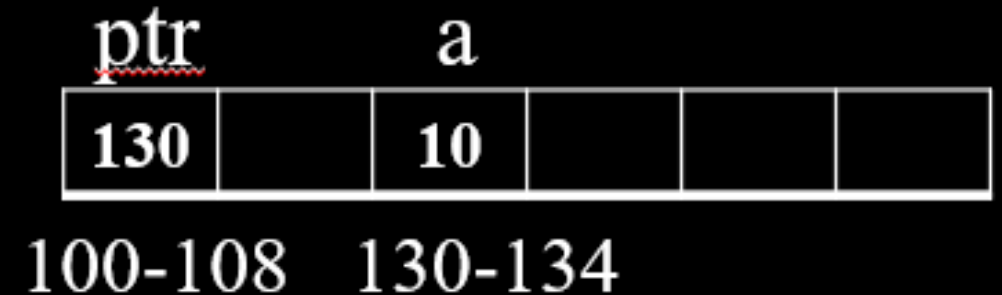
a		ptr2		ptr1	
A		250		200	
200		216-224		250-258	

Pointer arithmetics

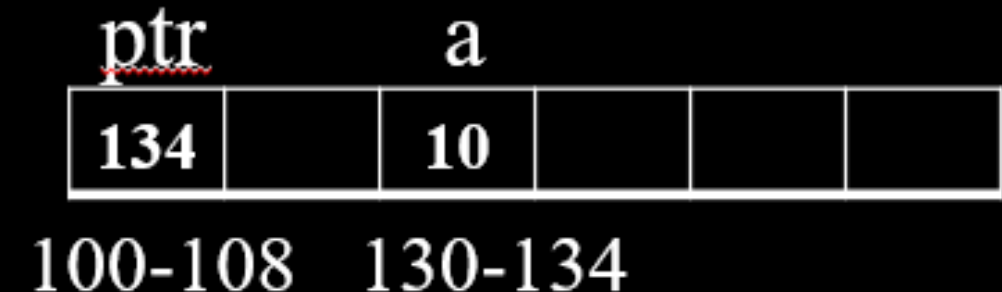
```
int a = 5;  
int *ptr = &a;
```



```
*ptr = *ptr + 5;
```



```
ptr = ptr + 1;
```



```
int arr[] = {10, 20, 30, 40, 50, 60, 70};  
int *ptr = arr;
```

```
printf("%d\n", *ptr);           // = ?  
printf("%d\n", *(ptr + 3));    // = ?  
printf("%d\n", ptr[2]);        // = ?  
printf("%ld\n", (ptr + 4) - ptr); // = ?
```

```
int matrix[3][4] = {  
    {1, 2, 3, 4},  
    {5, 6, 7, 8},  
    {9, 10, 11, 12}  
};
```

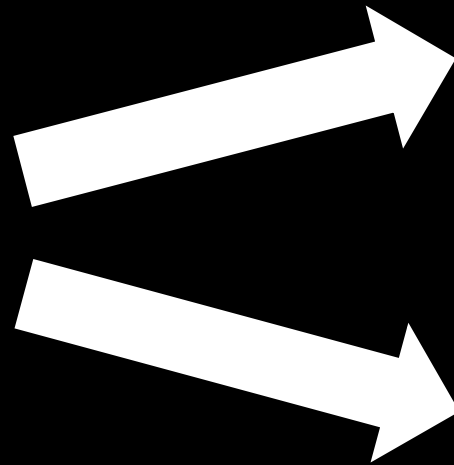
```
int (*ptr)[4] = matrix;
```

```
printf("%d\n", **ptr);           // = ?  
printf("%d\n", *(*ptr + 1) + 2); // = ?  
printf("%d\n", (*ptr)[3]);       // = ?  
printf("%d\n", *(ptr[2] + 1));   // = ?  
printf("%ld\n", (char*)(ptr + 2) - (char*)ptr); // = ?
```

Endianness (порядок байт)

- Big Endian (обратный порядок): Сначала старший байт, число слева направо
- Little Endian (прямой порядок): Сначала младший байт, число справа налево

`int a = 0x12345678`



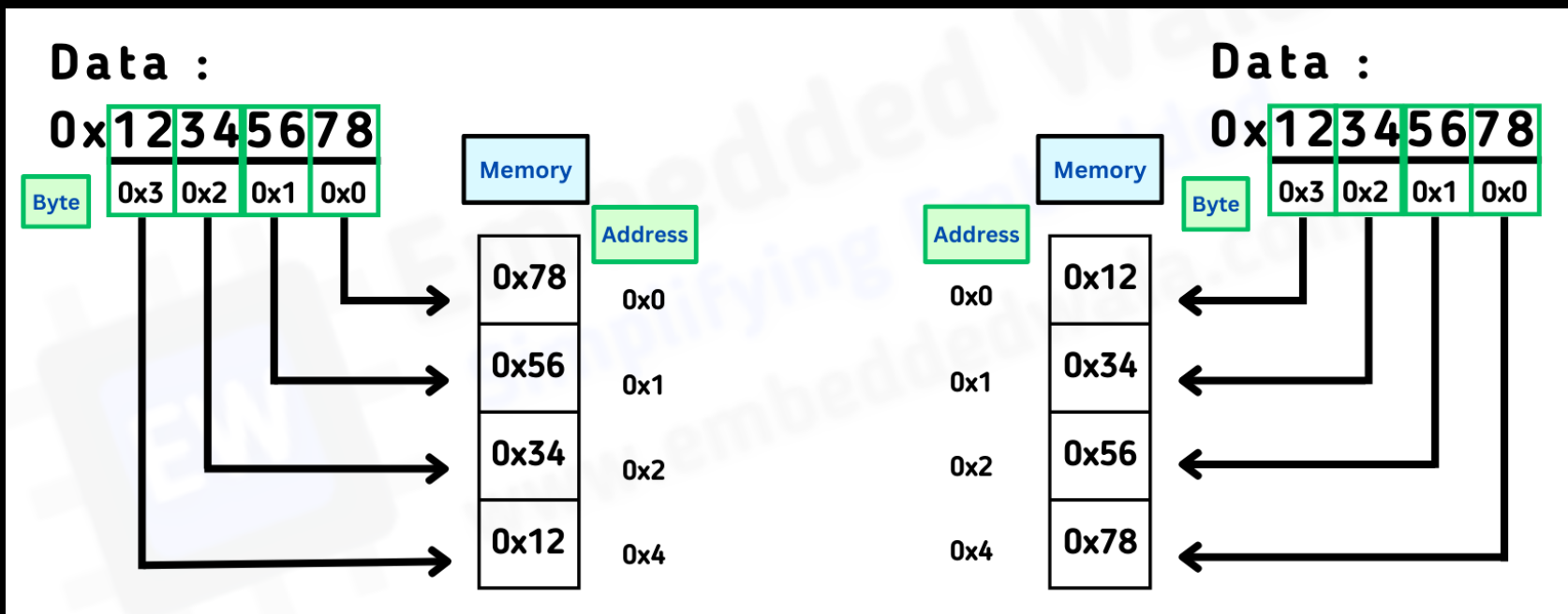
Big-endian

0x12	0x34	0x56	0x78
100	101	102	103

Little-endian

0x78	0x56	0x34	0x12
100	101	102	103

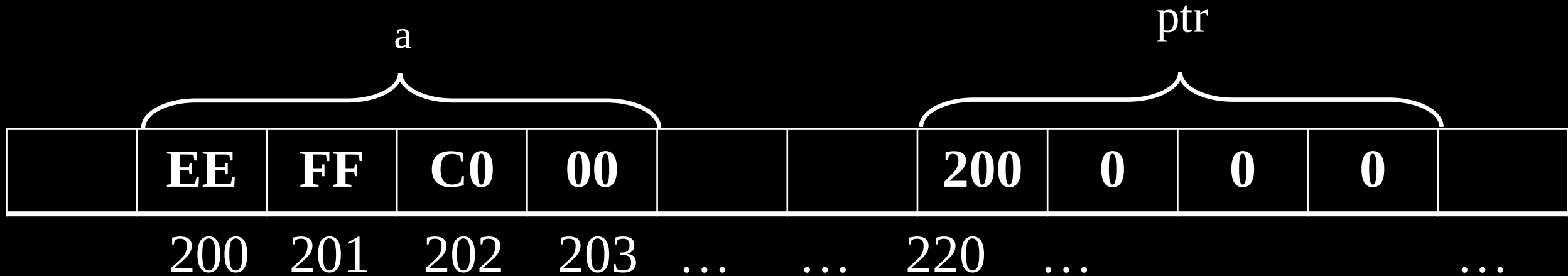
- В современных ПК с архитектурой x86 (Intel, AMD) вы увидите little-endian (прямой порядок байт)
- Сетевые протоколы строились на big endian, называется это сетевым порядком байт



Endianness проблема может возникнуть когда:

- Обмениваетесь данными между двумя системами с разным порядком байт
- Передача данных по сети (TCP/IP)
- Работа с файлами (PNG, JPEG)
- При чтении памяти


```
int a = 0xC0FFEE;  
char *ptr = (char *)&a;  
  
// для little-endian  
printf("first byte of a = %hhx\n", *ptr); // 0xEE  
ptr++;  
printf("second byte of a = %hhx\n", *ptr); // 0xFF  
ptr++;
```



Dynamic array

Динамические массивы - массивы, размер которых можно менять по ходу выполнения программы.

Функции для работы с динамическими массивами (<stdlib.h>):

- malloc()
- calloc()
- realloc()
- free()

malloc

malloc - Memory Alloc - выделяет динамическую память для хранения данных. Имеет 1 аргумент - количество байт, которое необходимо выделить. Возвращает адрес на выделенную память в случае успеха, в случае неудачи (кончилась память, например) - возвращает NULL.

```
int *arr = malloc(3*sizeof(int));

if (arr == NULL) {
    printf("Error malloc!\n");
    return -1;
}
```

malloc

```
int *arr = malloc(3*sizeof(int));  
  
if (arr == NULL) {  
    printf("Error malloc!\n");  
    return -1;  
}
```



malloc

```
arr[0] = 123; arr[1] = 2; arr[2] = 3;
```

// или

```
*arr = 123; *(arr + 1) = 2; *(arr + 2) = 3;
```



free

```
free(arr);
```



При попытке залезть в освобожденную память мы можем увидеть разные случайные числа от других программ, указатель на такую память называют "висячим указателем" (dangling pointer). В этом указателе остался адрес на память, которая уже не связана с нашей программой либо эта память достанется нам позже и при работе с этим указателем мы поймем лишь непредсказуемое поведение и "гейзенбаги" (см. в гугле).

free

```
free(arr);
```

```
arr = NULL;
```

arr

0						55555				321				0xffff12414				
100	101	102	103	...		800	801	802	803	804	805	806	807	808	809	810	811	

При повторной попытке разыменовать указатель на NULL мы гарантированно получим segmentation fault – а такое очень легко пофиксить отладчиком

Memory leak

Утечка памяти - ситуация, когда программа выделяет память и не освобождает, например:

```
int *ptr = malloc(10000);  
ptr = malloc(20000);
```

В случае если ваша програма завершилась, а вы не сделали free - то, **ОС подчистит всю память за вас**. Однако если ваша программа работает долго в цикле и в ней постоянно выделяется память и не освобождается..
В языке Си нет сборщика мусора

double free

```
int *ptr = malloc(100);  
free(ptr);  
free(ptr);
```

При попытке освободить память дважды, наша программа вылетит с ошибкой double free

calloc

calloc делает тоже самое что и **malloc**, но инициализирует данные нулями. Первый аргумент – количество элементов, второй количество байт для каждого элемента

```
int *arr = (int *)calloc(3, sizeof(*arr));  
// или (int *)calloc(1, 3 * sizeof(*arr));
```

С помощью malloc и memset можно добиться эффекта calloc

```
// 3 элемента по sizeof(int)
int *arr = malloc(3 * sizeof(int));

// начиная с адреса arr установить 12 нулей
memset(arr, 0, 3 * sizeof(int));
```

*calloc работает чуть медленнее чем malloc, т.к. он зануляет память, однако он в любом случае будет быстрее чем malloc + memset

realloc

```
char *str = malloc(N * sizeof(char));  
if (str == NULL) return -1;  
// ..  
// уменьшим массив до M байт  
str = realloc(str, M * sizeof(*str));
```

realloc позволяет перевыделять память, сохраняя при этом оригинальные значения в выделенной памяти.

*Если realloc не сможет перевыделить, то вернет NULL

Что если мы хотим создать динамическую матрицу?
Или 3-мерный дин. массив? 4-мерный и т.д.
Можно так, но..

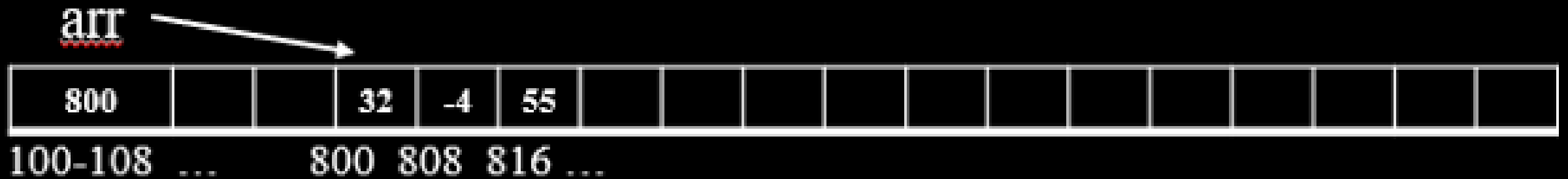
```
#define N 3
// ..
int *arr[N];
for (int i = 0; i < N; i++) {
    arr[i] = calloc(N, sizeof(arr[i]));
}
```

```
#define N 3
```

// ..

```
char **arr = malloc(N * sizeof(*arr));
```

```
if (arr == NULL) { /* error */ }
```



```

for (int i = 0; i < N; i++) {
    arr[i] = malloc(sizeof(**arr));
    if (arr[i] == NULL) { /* error */ }
    for (int j = 0; j < N; j++) {
        arr[i][j] = 'A' + i; // 'A', 'B'..
    }
}

```



```
arr[0][0] = 'X';  
arr[1][2] = 'Y';
```

// или

```
**arr = 'X';  
*(* (arr + 1) + 2) = 'Y';
```



```
void print_array(char **arr) {  
    for (int i = 0; i < N; i++) {  
        for (int j = 0; j < N; j++) {  
            printf("%c ", arr[i][j]);  
        }  
        printf("\n");  
    }  
}
```

```
// ..
```

```
print_array(arr);
```

```
// ..
```

```
for (int i = 0; i < N; i++) {  
    free(arr[i]);  
    arr[i] = NULL;  
}
```

```
free(arr);  
arr = NULL;
```

Для освобождения пройдемся в обратном порядке (относительно выделения памяти).

Почему освобождаем сначала `arr[i]`, а потом `arr`, а не наоборот?

Характеристика	Статический массив	Динамический массив
Выделение памяти	На этапе компиляции, в стеке (stack)	Во время выполнения, в куче (heap)
Размер	Фиксированный, известен на этапе компиляции	Может изменяться во время выполнения
Время жизни	Автоматическое (при выходе из области видимости)	Контролируется программистом
Освобождение памяти	Автоматическое	Ручное (с помощью <code>free()</code>)
Изменение размера	Невозможно	Возможно с помощью <code>realloc()</code>
Производительность	Быстрее (выделение в стеке)	Медленнее (выделение в куче)
Безопасность	Более безопасен (автоуправление)	Риск утечек памяти
Размер массива	<code>sizeof(arr)</code> возвращает общий размер	<code>sizeof(ptr)</code> возвращает размер указателя
Область видимости	Локальная или глобальная	Доступна откуда угодно, пока не освобождена

*Также по умолчанию размер в ОС, который мы можем создать для массива для динамической памяти не ограничен, а для стека ограничен (1-8МБ обычно, см. `ulimit -a` в командной строке linux, снимается ограничение этой же утилитой)

Variable scope

```
int a;
```

Глобальная область

```
int func(void) {  
    // ..  
}
```

Здесь **a** - **глобальная переменная**, т.к. она объявлена за всеми функциями (не входит ни в одно из тел функций), это значит, что переменная **a** будет видна всем функциям. Существует глобальная переменная всю жизнь программы.

Variable scope

```
int func(int a) {  
    int b;  
    // ..  
}
```

Локальная область

Здесь *a* - аргумент в ф-ию, видна только внутри этой функции, тоже самое касается и локальной переменной с именем *b*.

Variable scope

```
int func(int a) {  
    // ..  
    {                Блок  
        int b;  
    }  
    // ..  
}
```

b - локальная переменная для блока в фигурных скобках {},
будет видна только внутри этих {}

```
#include <stdio.h>

int a = 15;

void func(void) {
    printf("global a = %d\n", a);
}

int main(void) {
    int a = 10;
    {
        int a = 20;
    }
    printf("main a = %d\n", a);
    func();
}
```

Stack vs Heap

- **Stack (стек)** - это быстрая и упорядоченная область памяти, работающая по принципу LIFO (Last In First Out, последний пришел — первый ушел).
 - Это как стол, на котором лежит стопка книг, для доступа к нужной книге сначала достанем все книги сверху. Если книг положить очень много на стек - то они свалятся со стола, называется это `stack overflow` (переполнение стека), такое можно сделать создав массив больше размера стека, либо сделав бесконечную рекурсию..
- **Heap (куча)** - это большая и гибкая область памяти, где можно выделять блоки произвольного размера вручную. Управление памятью лежит на программисте (или в некоторых ЯП сборщике мусора, на англ. `garbage collector`)
 - Это как гараж, места много, но все валяется хрен знает где, поэтому найдем что ищем не так быстро

Stack vs Heap

Критерий	Stack (Стек)	Heap (Куча)
Основное назначение	Локальные переменные, вызовы функций	Динамическое выделение памяти
Принцип работы	LIFO (Last In, First Out)	Свободное распределение
Управление памятью	Автоматическое	Ручное (malloc/calloc/realloc/free)
Скорость доступа	⚡️ быстренько	🐢 не очень
Риски	Stack Overflow	Утечки памяти
Время жизни данных	Пока выполняется блок тела	Пока явно не освобождена
Идеально для	Временных данных	Объектов с заранее неизвестным размером

Есть такой форум для программистов, в который вы вероятно попадете если ИИ не даст ответ на ваш вопрос и вы пойдете в гугл

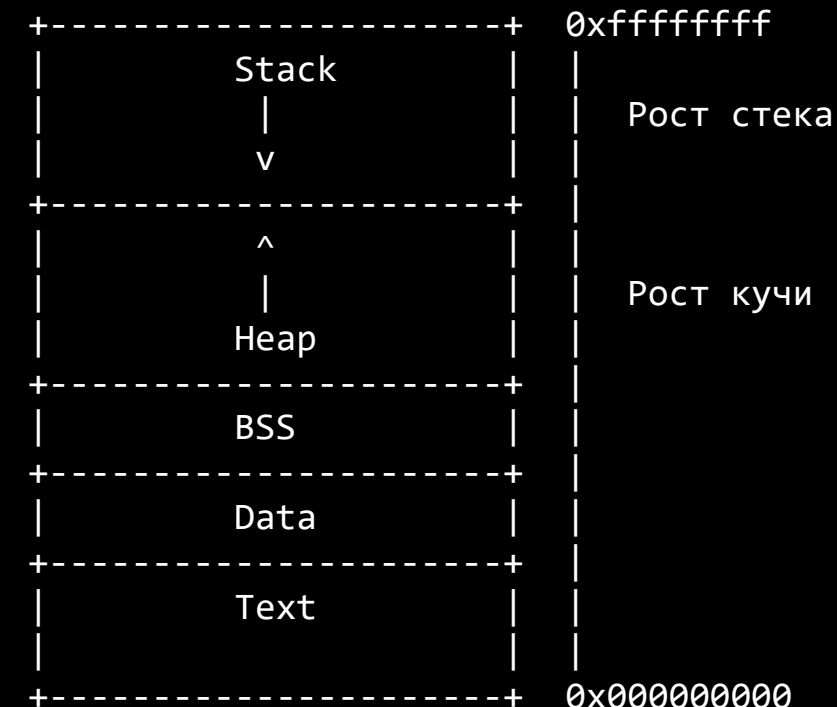


Модель памяти языка Си

Модель памяти - абстракция, с помощью которой описывается взаимодействие программы с памятью.

Любую программу на языке Си в памяти можно представить так следующих секций:

1. Text - Код программы (текст), здесь все наши функции с кодом
2. Data - Данные, здесь хранятся все переменные явно проинициализированные чем-либо кроме 0
3. BSS (Block Started by Symbol) - все глобальные переменные, что инициализированы 0 или не инициализированы ничем
4. Heap - секция кучи, вся динамически выделенная память через malloc/calloc/realloc
5. Stack - секция стека, все локальные переменные, вызовы функций (при вызове функции создается кадр функции для возможности вернуться в вызываемое место)



Модель памяти языка Си

model.c

```
#include <stdio.h>
#include <stdlib.h>

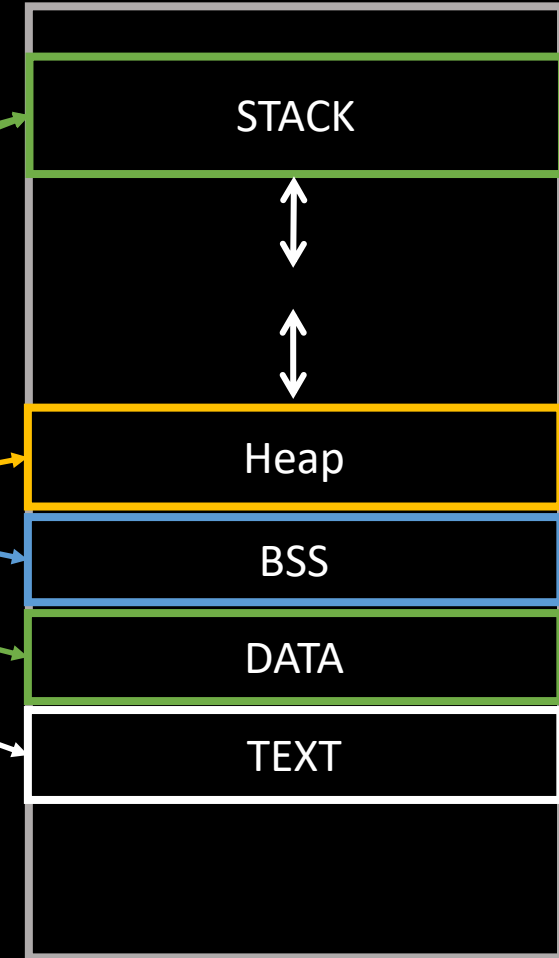
int a;
int b = 7;

int main(void) {
    double c;
    short d = 1;

    void *ptr = NULL;
    ptr = malloc(20);

    printf("a = %d, b = %d, c = %lf, d = %hd, ptr = %p\n",
           a, b, c, d, ptr);
}
```

0xFFFFFFFF



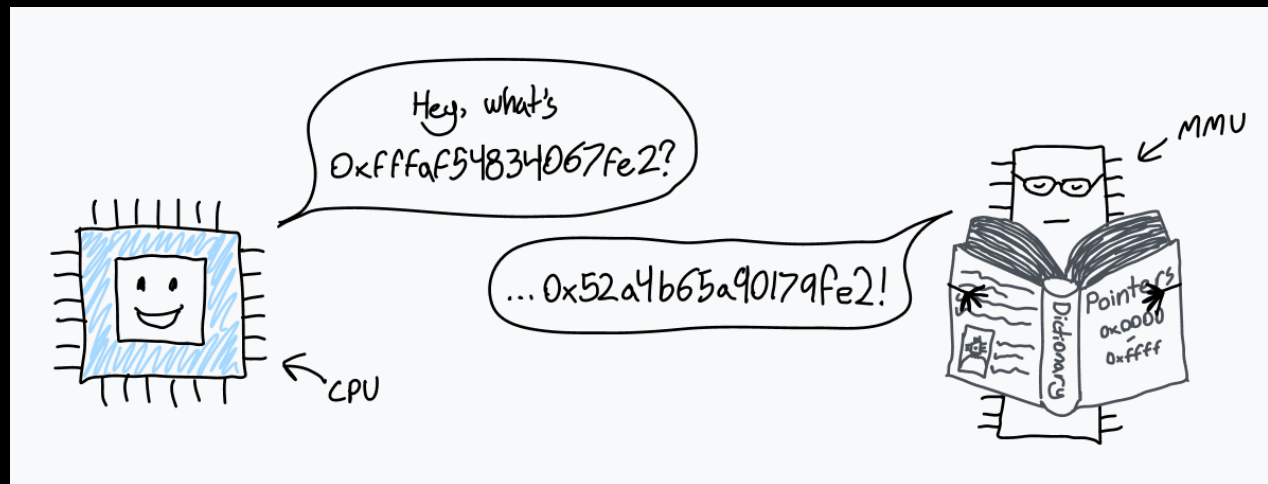
0x00000000

Допустим у нас есть память и программа А и Б

- Что если программа А захочет залезть в память программы Б? Она это возьмет и сделает без проблем, т.е. выйдет так, что любая программа способна положить всю операционную систему (ОС - тоже программа) в случае неправильной работы с памятью
- Что если оперативная память у нас на 8 ГБ, программа А занимает 6 ГБ памяти, а программа Б 3 ГБ памяти? А если еще и запустить программу С? Придется ждать пока программа А завершится, чтобы программа Б влезла

Virtual and Physical memory

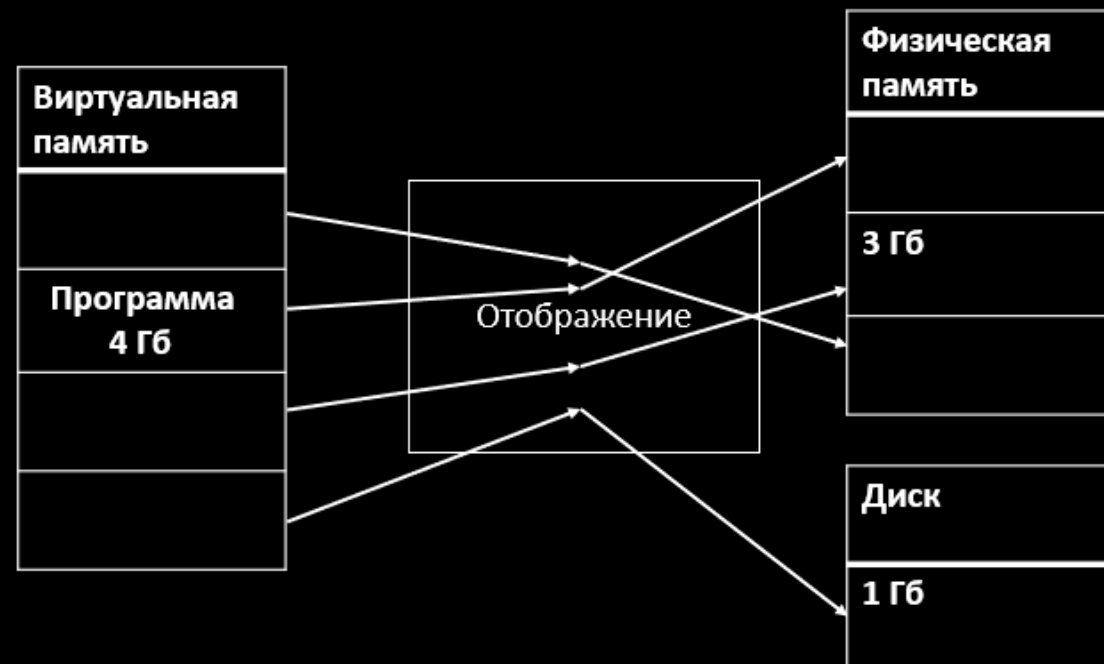
Виртуальная память как и физическая разбиваются на блоки фиксированного размера - **страницы** (обычно 4КБ). Когда программа пытается получить доступ к своей виртуальной памяти, **ОС и процессор и чип MMU (Memory Management Unit) переводят виртуальный в физический адрес**. Когда процу говорят прочитай значение целого числа по некоторому адресу, то он лезет в MMU для перевода этого адреса в физический и далее получив адрес читает что надо из оперативки.



<https://cpu.land/the-translation-in-your-computer#memory-is-fake>

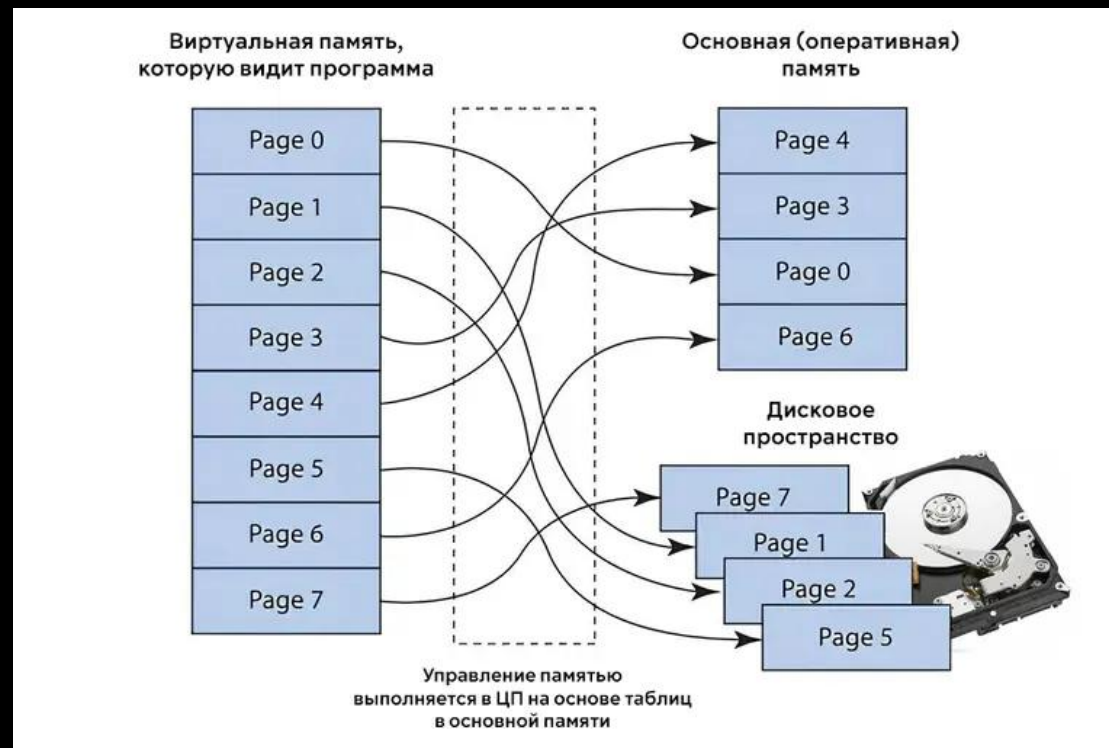
Virtual and Physical memory

Не всем страницам обязательно быть в оперативной памяти, некоторые неиспользуемые можно временно сгрузить на более медленные хранилища (SSD или HDD) в специальный **файл подкачки** (swap-раздел). Так, например, можно открыть Google Chrome и в нем кучу-кучу вкладок, рано или поздно эти вкладки съедят всю оперативку и начнут сгружаться на жесткий диск, а если мы попробуем открыть старую вкладку, то она подкачается с HDD в оперативку и произойдет это не очень быстро. Таким образом мы можем запустить программ больше чем влезет в оперативную память



Virtual and Physical memory

С виртуальной памятью у каждой программы есть свое адресное пространство, так как у каждой программы свое отображение виртуальных на физические адреса и при попытке залезть в виртуальные адреса, что не размаплены на физически мы получим segmentation fault.



Github

<https://github.com/kruffka/C-Programming>

